
What Are Requests and Limits in Kubernetes?

In Kubernetes, every container runs within a **Pod**, and each container needs **resources** like CPU and memory to run. Kubernetes allows you to declare how much of these resources each container needs and is allowed to use, via:

- **Requests** : What the container *needs* to run reliably.
- **Limits** : The *maximum* the container is *allowed* to use.

These configurations are declared in the pod spec under **resources**, and they help Kubernetes with **efficient scheduling**, **stability**, and **fairness** across applications.

What Are Requests?

Definition:

A **request** is the **minimum guaranteed amount** of CPU or memory that the container is assured to receive when scheduled.

 This ensures:

- The **Kubernetes scheduler** knows how much a pod needs.
- The pod won't be placed on a node that doesn't have enough available resources.

 If a node has 2 CPU cores and a pod requests 1 core, only one more pod requesting 1 core can be scheduled there.

CPU Request Example:

resources:

requests:

cpu: "500m"

This means:

- The container needs **500 milliCPU**, or **0.5 CPU cores**.
- Kubernetes will schedule the pod on a node with at least 0.5 CPU available.

 Note: CPU is a **compressible resource**, meaning it can be **shared** among containers, but at the risk of **throttling** later if a limit is set.

Memory Request Example:

resources:

requests:

memory: "512Mi"

This means:

- The container will be guaranteed **512 MiB of RAM**.

 Memory is a **non-compressible resource**—a container *must* have at least this amount or it will not run.

What Are Limits?

Definition:

A **limit** is the **maximum amount** of CPU or memory a container is allowed to use.

 Kubernetes uses limits to:

- Protect nodes from resource overuse.

- Prevent one container from starving others.
 - Kill or throttle containers that exceed their limits.
-

⚙️ CPU Limit Example:

resources:

limits:

cpu: "1"

This means:

- The container can't use more than **1 full CPU core**.
- If it tries, it gets **throttled** 🐢 (slowed down), not killed.

📊 CPU throttling affects performance but keeps the container alive.

💣 Memory Limit Example:

resources:

limits:

memory: "1Gi"

This means:

- The container is allowed to use **up to 1 GiB** of memory.
- If it uses more → it is **terminated with an OOMKilled error** ❌💣.

💡 Unlike CPU, memory can't be "throttled"—it's all or nothing.

Requests vs Limits (Full Comparison)

Feature	Requests 💡	Limits 🚫
Scheduler Uses It?	✅ Yes	❌ No
Guaranteed Resource?	✅ Yes	❌ No
Max Usage Capped?	❌ No (can burst)	✅ Yes
Memory Overuse Result	May get extra (if no limit)	OOMKilled 💥
CPU Overuse Result	May burst	Throttled 🐢
Prevent Resource Abuse	❌ Not enforced	✅ Enforced

Real-World Scenario

Imagine you're deploying a Node.js API server. You know it usually runs fine with **256Mi** of RAM and **0.25 CPU**, but under load, it might spike to **512Mi** and **0.5 CPU**.

resources:

requests:

memory: "256Mi"

cpu: "250m"

limits:

memory: "512Mi"

cpu: "500m"

 What does this do?

- Guarantees it always gets **what it needs**.
 - Prevents it from **hogging resources** during spikes.
-

💣 What Happens If You Don't Set Requests or Limits?

- No **requests** → Pod might be scheduled to a busy node and underperform 😞
 - No **limits** → Pod can consume too much memory/CPU and crash the whole node 😱
 - **Only requests** → Safe scheduling, but resource abuse possible 🌀
 - **Only limits** → May run poorly if it can't get the resources it needs 🤔
-

🛡️ Best Practices

✅ Always set both **requests and limits**



Use monitoring tools like **Prometheus** and **Grafana** to tune values



Use **Horizontal/Vertical Pod Autoscalers** to auto-scale based on usage



Set **ResourceQuotas** in namespaces to control cluster-wide usage



Use **default limit ranges** in namespaces for fallback protection
